Clinical Workflow Intelligence Platform

Complete Project Specification and System Design

Purpose: A resume-aligned healthcare software project that supports clinical information review without duplicating brain-imaging analysis or patient-treatment products.

1. Executive Summary

The Clinical Workflow Intelligence Platform is a proposed supporting application for healthcare teams. It helps authorized users organize clinical documents, retrieve relevant information, ask grounded questions, review AI-generated summaries, and export approved information. The system does not diagnose diseases, analyze MRI images, generate brain maps, or replace a clinician.

Core idea: Patient or clinical information is uploaded, processed, indexed, retrieved through semantic search, and presented with source references.

2. Problem Statement

- Clinical information may be distributed across patient records, previous reports, doctor notes, and uploaded documents.
- Healthcare staff may spend time searching documents manually for relevant historical information.
- General-purpose AI can produce unsupported answers if it is not grounded in trusted source material.
- A lightweight system is needed to connect structured patient data with searchable clinical documents.

3. Proposed Solution

The platform combines a Flask backend, SQL database, document-processing pipeline, FAISS vector search, and Google Gemini. The application retrieves relevant document sections before generating an answer. Each response includes source references so the user can verify the information.

4. Project Boundaries

Included	Not included
Clinical document upload and storage	MRI/fMRI image analysis
Patient and document metadata	Brain mapping or connectomics
Semantic search over uploaded documents	Disease diagnosis
Grounded AI summaries and Q&A	Autonomous medical decisions
Review, approval, and export	Replacement for a clinician

5. Resume Skill Mapping

Resume skill	Project usage
Python	Backend logic, processing, validation, AI pipeline
Flask	REST APIs and server-side application
HTML5 / JavaScript	User interface and interactive screens

Google Gemini	Grounded summaries and question answering
RAG	Retrieval of relevant document context before generation
SQL	Users, patients, documents, reports, and audit records
FAISS	Vector similarity search
NumPy / Pandas	Data transformation and processing
Testing / Debugging	API validation and reliability checks
Git / GitHub	Version control and collaboration
Hugging Face	Optional deployment or demonstration hosting

6. End-to-End Architecture

```
FRONTEND
HTML5 + JavaScript
Login → Patient List → Patient Details → Upload Documents
↓
Ask AI Questions
↓
Review / Export
|
HTTP / REST
↓
FLASK BACKEND
Authentication and Authorization
Patient Management
Document Upload and Validation
Document Processing
RAG Pipeline
Gemini Integration
Report Export
|
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
↓ ↓
SQL DATABASE RAG PIPELINE
Users, Patients, Documents Text Extraction → Chunking
Reports, Audit Logs ↓
Embeddings
↓
FAISS Vector DB
↓
Similarity Search
↓
Relevant Context
↓
Gemini API
↓
Grounded Response
```

7. Main Modules

- **Authentication and authorization:** Controls access to the application and ensures only authorized users can access records.
- **Patient management:** Creates and manages patient metadata and links documents to a patient.
- **Document management:** Uploads, validates, stores, and lists clinical documents.
- **Document processing:** Extracts text, cleans it, and divides it into searchable chunks.
- **Vector indexing:** Creates embeddings and stores them in FAISS.
- **Question-answering:** Retrieves relevant chunks and sends grounded context to Gemini.
- **Review and approval:** Allows users to review AI output before treating it as an approved summary.
- **Export:** Generates a structured PDF or downloadable report.
- **Audit logging:** Records important actions such as upload, search, review, and export.

8. Detailed Workflow

- **1. User login:** The user signs in and the backend verifies credentials and permissions.
- **2. Create or select patient:** The user opens an existing patient record or creates a new one.
- **3. Upload document:** The frontend sends a document to Flask using a multipart form request.
- **4. Validate and store:** The backend checks file type, size, ownership, and metadata before saving it.
- **5. Extract text:** Python extracts text from the document and records processing status.
- **6. Chunk text:** The extracted content is split into smaller overlapping sections.

- **7. Generate embeddings:** Each chunk is converted into a vector representation.
- **8. Index in FAISS:** The vectors are stored with metadata that points back to the source document.
- **9. Ask a question:** The user enters a question about the selected patient's documents.
- **10. Retrieve context:** The system embeds the question and retrieves the most similar chunks.
- **11. Generate answer:** Gemini receives the question, retrieved context, and safety instructions.
- **12. Display sources:** The frontend shows the answer with document names and relevant sections.
- **13. Review and export:** The user can edit, approve, and export the final information.

9. RAG Pipeline

Retrieval-Augmented Generation reduces unsupported answers by retrieving relevant source content before calling the language model.

Stage	Description
Ingestion	Receive and validate uploaded documents.
Extraction	Convert document content into plain text.
Chunking	Split text into manageable sections with overlap.
Embedding	Convert each section into a numerical vector.
Indexing	Store vectors in FAISS and retain source metadata.
Query embedding	Convert the user question into a vector.
Similarity search	Retrieve the most relevant document chunks.
Prompt construction	Combine question, context, and response rules.
Generation	Ask Gemini to answer only from the supplied context.
Citation mapping	Attach source document and section references.

10. Suggested Database Schema

Table	Important fields
users	id, name, email, password_hash, role, created_at
patients	id, patient_code, display_name, date_of_birth, created_at
documents	id, patient_id, filename, file_path, document_type, status, uploaded_by, created_at
document_chunks	id, document_id, chunk_index, text, vector_reference
reports	id, patient_id, question, answer, sources, status, reviewed_by, created_at
audit_logs	id, user_id, action, resource_type, resource_id, timestamp

11. REST API Design

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate user
GET	/api/patients	List accessible patients
POST	/api/patients	Create patient record
GET	/api/patients/{id}	Get patient details
POST	/api/patients/{id}/documents	Upload document
GET	/api/patients/{id}/documents	List patient documents
POST	/api/documents/{id}/process	Process and index document
POST	/api/patients/{id}/ask	Ask grounded question
POST	/api/reports/{id}/approve	Approve reviewed report
GET	/api/reports/{id}/export	Export report

12. Frontend Screens

- Login screen
- Dashboard with patient and document counts
- Patient list with search and filters
- Patient details page with document timeline
- Document upload and processing status
- AI question-and-answer panel
- Source-reference panel
- Report review and approval screen
- Export/download screen

13. Example Demo Scenario

Input: A user uploads a sample clinical report named patient_report_01.pdf.
Question: “Summarize the important findings from the previous report.”
System behavior: The application retrieves the most relevant sections, sends them with the question to Gemini, and displays a concise answer with the source filename and section references.
Expected result: The user can verify the answer against the uploaded document before approving or exporting it.

14. Safety and Reliability

- Use synthetic or de-identified data for the prototype.
- Display a clear notice that the system is a decision-support prototype, not a diagnostic tool.
- Do not allow the model to invent missing clinical facts.
- If the answer is not supported by retrieved context, return: “The uploaded documents do not contain enough information to answer this.”
- Restrict access to patient records using authentication and authorization.
- Validate file type, file size, and uploaded content.
- Log important actions for traceability.
- Require human review before exporting an approved report.

15. Recommended Folder Structure

```
clinical-workflow-platform/
■■■ app.py
■■■ config.py
■■■ requirements.txt
■■■ .env
■■■ README.md
■■■ routes/
■ ■■■ auth_routes.py
■ ■■■ patient_routes.py
■ ■■■ document_routes.py
■ ■■■ report_routes.py
■■■ services/
■ ■■■ document_processor.py
■ ■■■ embedding_service.py
■ ■■■ vector_store.py
■ ■■■ rag_service.py
■ ■■■ gemini_service.py
■■■ models/
■ ■■■ user_model.py
■ ■■■ patient_model.py
■ ■■■ document_model.py
■ ■■■ report_model.py
■■■ templates/
■ ■■■ login.html
■ ■■■ dashboard.html
■ ■■■ patients.html
■ ■■■ patient_detail.html
■■■ static/
■ ■■■ css/
■ ■■■ js/
■■■ uploads/
■■■ vector_store/
■■■ tests/
■■■ test_auth.py
■■■ test_documents.py
■■■ test_rag.py
```

16. Development Plan

Phase	Deliverable
Phase 1	Set up Flask project, environment variables, Git, and basic UI.
Phase 2	Implement authentication and SQL patient management.
Phase 3	Implement document upload, validation, and storage.
Phase 4	Implement text extraction and chunking.
Phase 5	Implement embeddings and FAISS indexing.
Phase 6	Integrate Gemini with grounded prompts.
Phase 7	Build source references, review, and export.
Phase 8	Add testing, error handling, documentation, and deployment.

17. CTO Explanation

“I wanted to build a healthcare application that complements existing clinical AI systems rather than duplicating them. My project focuses on the information workflow around clinical documents. I used Flask for the backend, SQL for structured data, FAISS for semantic search, and Gemini for grounded question answering. The main challenge was making sure the AI answers were based on retrieved document context rather than generating unsupported information. I also implemented document processing, API validation, review workflows, and testing.”

18. Final Positioning

This project is intentionally designed as a supporting clinical-information platform. It demonstrates practical software engineering, AI integration, RAG, database design, API development, and reliability practices using skills already present on the resume. It should be presented as an independent prototype inspired by healthcare workflow challenges—not as an existing BrainSightAI product and not as a replacement for VoxelBox or Snowdrop.

Prepared as a project planning document. Use synthetic or de-identified data for demonstrations.